

Practical Techniques in Fundamental Software Development: Intelligent Software Engineering

Intelligent Software Engineering Experiment Report

Experiment 1: Mining Code Review Comments and Understanding Requirements

Name	Marlen Melis	Department	School of Computing
Student ID	2026140428	Course Teacher	Liu Jiakun
Instructor	Liu Jiakun	Lab Location	Dormitory
Lab Time	Evening		
Lab Score		Discussion Score	
Report Score		Total Experiment Score	

1 Experiment Objectives

Upon completion of this experiment, I am able to:

1. Understand the basic workflow of Pull Requests (PRs) and code reviews in modern software development (fork → commit → open PR → review → discuss → merge/close).
2. Explain how code review data is organized inside a real GitHub open-source repository, across five actively-maintained projects spanning three languages.
3. State the relationships between Pull Requests, Commits, Reviews, Review Comments, Issue Comments, and Labels, and represent them as seven linked flat tables.

4. Retrieve code review data at scale using the authenticated GitHub GraphQL and REST APIs, with on-disk caching for idempotent, resumable mining.
5. Extract multi-source information — code diffs, commit messages, PR text, and structured metadata — for every mined Pull Request.
6. Apply statistical analysis and visualization to the resulting dataset, and use it to establish an explicit, evidence-based data foundation (including a documented class-imbalance report) for the machine-learning and LLM experiments that follow.

2 Experiment Content

1. Repositories and Sampling Strategy

Five large, actively-maintained open-source repositories were selected, chosen so that each had a verifiable population of AI-authored Pull Requests (queried live against GitHub’s search API before finalizing the shortlist, rather than assumed): `microsoft/vscode` (TypeScript), `dotnet/runtime` and `dotnet/aspnetcore` (C#), `microsoft/semantic-kernel` (Python/C#), and `home-assistant/core` (Python).

A pure random sample of 300 closed PRs per repository would contain few or zero AI-authored PRs by chance, since AI-authored PRs are a small minority of all PRs in most repositories. The sampling strategy therefore, per repository: (1) searched for PRs authored by GitHub’s own Copilot coding agent (`is:closed author:app/copilot-swe-agent`), capped at 100; (2) filled the remaining slots with a sample of recent closed PRs, skipping any overlap with step 1. This deliberately over-represents AI-authored PRs relative to their natural base rate — a documented design decision, not a hidden shortcut. Only closed PRs were targeted, so every sampled PR has a final merge/close label.

2. Mining Pipeline

An authenticated GitHub client (`src/mining/github_client.py`) combines one nested GraphQL query per PR — core fields, commits, reviews and their inline comments, top-level issue comments, and labels — with paginated REST calls for file-level diff patches, since GraphQL exposes no unified-diff field. Every raw PR response is cached to disk as JSON, keyed by (`owner`, `repo`, `number`), making the mining job idempotent and resumable: re-running only fetches PRs still missing from the cache. Requests are retried with exponential backoff and jitter on HTTP 403/429/5xx, and separately on GraphQL-level rate-limit errors (which GitHub returns as HTTP 200 with an `errors` array, not an

HTTP error code). PRs are fetched concurrently via a thread pool (10 workers), giving roughly an order of magnitude better throughput than sequential fetching.

3. AI-Authorship and AI-Reviewer Detection

Detection (`src/mining/ai_detection.py`) uses a three-tier signal hierarchy rather than a username heuristic:

1. **Structural primary signal:** GitHub’s GraphQL API tags every actor with `__typename` of `Bot` or `User` directly.
2. **Semantic sub-classification of Bot actors:** Bot-typed actors are matched against curated allowlists of AI coding agents (e.g. `copilot-swe-agent`) and AI review bots (e.g. `copilot-pull-request-reviewer`, `coderabbitai`). Non-AI automation (`dependabot`, `renovate`, `github-actions`, ...) is excluded explicitly; anything Bot-typed but on neither AI list is reported honestly as `other_bot` rather than guessed into an AI category.
3. **Secondary, lower-confidence signal:** for PRs opened by a human whose commit trail still shows AI involvement — an agent commit identity/no-reply email pattern, or Copilot’s literal “Initial plan” placeholder first commit — flagged as a distinct `human_opened_ai_assisted` category at explicitly lower confidence than a fully agent-authored PR.

Every detection carries a `method`, a `confidence`, and human-readable `evidence`, so it can be spot-checked rather than trusted blindly. A manual precision review was performed on 59 distinct flagged PRs (a random sample of 40, plus a full census of all 19 PRs flagged only by the weaker secondary signal); zero false positives were found (Section “Experimental Results” below).

4. Data Model and Statistical Analysis

Mined PRs are flattened into seven parquet tables — `pull_requests`, `commits`, `files_changed`, `reviews`, `review_comments`, `issue_comments`, `ai_signals` — with AI-detection flags folded directly onto `pull_requests` as well as kept in their own table. `src/mining/analyze_exp1.py` computes descriptive statistics and generates visualizations for every measure the lab guide asks for — merge status, review-comment-count distribution, reviewer-count distribution, PR-length distribution, label distribution, AI-vs-human PR counts, and AI-reviewer presence — broken out *per repository and pooled*, since a pooled-only number can hide very different per-repository behavior.

3 Experimental Results

Requirement: Describe the results obtained from the experiment. Basic content should include:

1. Samples of Successfully Retrieved Pull Request Data

Repo	#	Title	Author	Merged	+li
microsoft/vscode	326979	dictation: publish native run-time to CDN instead of...	meganrogge (human)	Yes	1
microsoft/vscode	326697	Fix Cmd/Ctrl+I dictation in the Agents window composer	copilot-swe-agent (AI)	Yes	

Every retrieved PR carries: GraphQL node ID, repo, number, title, body, author login/type, timestamps, state, merge status, additions/deletions/changed-files counts, labels, and the full AI-detection verdict (`is_ai_authored`, `ai_authored_kind`, `ai_authored_method`, `ai_authored_confidence`, `is_ai_reviewed`).

2. Samples of Successfully Retrieved Review Comments

Human reviewer on `tests/components/caldav/test_calendar.py`:

“Thanks for the feedback! I’ll remove it.”

AI reviewer (copilot-pull-request-reviewer) on `homeassistant/components/midea_lan/co`

“Guard against missing `DEFAULT_CLOUD` in the returned cloud server mapping to avoid `StopIteration` and a crashed config flow.”

Human reviewer on `homeassistant/components/denon_rs232/media_player.py`:

“Will be modified, along with the same issue at line 79 (for consistency).”

Even from three examples, a qualitative pattern is visible and is discussed further under Reflection Question 4: the AI reviewer’s comment is a single, self-contained technical observation with a concrete failure mode named (`StopIteration`), while the human comments are conversational turns embedded in a back-and-forth thread.

3. Samples of Successfully Preprocessed Data

Raw GraphQL/REST responses were flattened into seven linked, parquet-serialized tables:

Table	Rows	Key
pull_requests	1,494	id (also carries AI-signal flags)
commits	6,861	pr_id
files_changed	27,110	pr_id
reviews	7,016	pr_id, review_id
review_comments	6,351	pr_id, review_id, comment_id
issue_comments	3,183	pr_id, comment_id
ai_signals	1,494	pr_id

Preprocessing included: normalizing labels from a nested GraphQL connection into a flat list per PR; de-duplicating PR numbers that appeared twice due to pagination drift on a live, frequently updated repository listing (one such case was found and fixed — see “Summary of Problems Encountered”); and folding the AI-detection verdict directly onto each PR row.

4. Statistical Results

Dataset size. 1,494 of 1,500 targeted PRs were successfully retrieved. The 6-PR shortfall has two distinct causes, both documented rather than silently dropped: **5 PRs** were permanently excluded after exhausting a 6-attempt retry budget on server-side 502 errors (`aspnetcore` 1, `runtime` 1, `semantic-kernel` 3), and **1 further slot** was lost in `microsoft/vscode` because pagination drift put the same PR on two pages, so that repo’s 300-entry sample list held only 299 *distinct* PRs (see “Summary of Problems Encountered” #2). Per repository: `dotnet/aspnetcore` 299, `dotnet/runtime` 299, `home-assistant/core` 300, `microsoft/semantic-kernel` 297, `microsoft/vscode` 299.

Merged vs. non-merged (class balance). Overall: **1,075 merged / 419 not merged (72.0% merge rate)** — a moderate class imbalance. Per repository the rate ranges from 63.0% (`semantic-kernel`) to 81.9% (`dotnet/runtime`), so the imbalance is not uniform across repositories either. This is discussed directly in Reflection Question 5.

Repo	Merged	Not merged	Merge rate
dotnet/aspnetcore	201	98	67.2%
dotnet/runtime	245	54	81.9%
home-assistant/core	219	81	73.0%
microsoft/semantic-kernel	187	110	63.0%
microsoft/vscode	223	76	74.6%
Pooled	1,075	419	72.0%

Review-comment-count distribution. Pooled mean 4.25, median 1, heavily right-skewed (std 12.4, max 242 on a single `home-assistant/core` PR); **701 of 1,494 PRs (46.9%) received zero inline review comments.** `home-assistant/core` has both the highest mean (7.13) and the most extreme outlier; `microsoft/vscode` the lowest mean (1.77).

Reviewer-count distribution. Pooled mean 2.46 distinct reviewers per PR, median 2, max 10; 172 PRs (11.5%) had zero reviewers recorded.

PR-length distribution (additions + deletions). Pooled median 57 lines — most PRs are small — but the mean (17,592) is dominated by extreme outliers (max 14,285,506 lines, almost certainly a vendored-dependency or lockfile-regeneration commit); the 90th percentile is only 702 lines. Per-repo medians are similar (36–72 lines) despite very different means, confirming the skew is driven by a small number of huge PRs, not a systematically different typical PR size across repos.

Label distribution. 1,071 of 1,494 PRs (71.7%) carry at least one label. The pooled top labels are dominated by `home-assistant/core`’s heavy labeling conventions (`cla-signed` 261, `small-pr` 169, `has-tests` 129), while `microsoft/vscode` uses labels far more sparingly (top label `dependencies` at only 12) — label usage is a per-repository convention, not a universal signal.

AI-authored vs. human-authored PRs. Pooled: **380 AI-authored (25.4%) vs. 1,114 human-authored**, but this share varies sharply by repo: 37.5% in `aspnetcore` and 36.8% in `dotnet/runtime` vs. only 10.3% in `home-assistant/core` and 7.7% in `semantic-kernel` — reflecting how actively each project’s maintainers have adopted the Copilot coding agent, not an inherent property of the codebase. Of the 380 AI-authored PRs, 361 (95.0%) were detected via the high-confidence allowlist signal (agent account, Bot-typed) and 19 (5.0%) via the lower-confidence secondary commit-trail signal — a genuinely distinct “human-opened, AI-assisted” category, close to the ~8% anticipated

at design time.

AI-reviewer presence. Pooled: **71.2% of PRs (1,064/1,494) had at least one AI reviewer touch them** — `copilot-pull-request-reviewer` is present on *all* 1,064 of them, with `coderabbitai` co-occurring on 4 (never alone: 1,060 PRs have Copilot only, 4 have both). Presence ranges from 42.8% (`semantic-kernel`) to 86.0% (`microsoft/vscode`).

Merge rate, AI-authored vs. human-authored. Pooled: AI-authored PRs merge at **65.3%** vs. human-authored at **74.2%** — but the direction is *not* consistent across repositories: in `dotnet/aspnetcore` (68.8% vs. 66.3%) and `microsoft/semantic-kernel` (69.6% vs. 62.4%), AI-authored PRs merge *more* often than human-authored ones, while in `home-assistant/core` the gap is large and in the opposite direction (35.5% vs. 77.3%). Per-repo sample sizes for AI-authored PRs are small in some repos (`semantic-kernel` $n = 23$, `home-assistant/core` $n = 31$), so these per-repo rates are noisier than the pooled figure and should be read with that caveat.

Manual precision spot-check. A random sample of 40 flagged PRs plus a full census of the 19 secondary-signal PRs (59 distinct PRs, 68 individual flag assessments) were manually reviewed against author/reviewer identity, GitHub’s structural typing, and — for the secondary-signal cases — the raw commit trail. **Zero false positives were found** (point-estimate precision 100%; $\geq \sim 92.5\%$ at 95% confidence by the rule of three, computed on the 40 random draws — the 19-PR census is a full census of the weakest tier, not a random sample, so it cannot be used to widen that bound). One labeling nuance was found: a Copilot-authored PR with a deleted (null) author account was correctly flagged `is_ai_authored=True` but mis-categorized as `human_opened_ai_assisted` rather than “author unknown” — an honest, reportable edge case, not an authorship error.

5. Results of the Visual Analysis

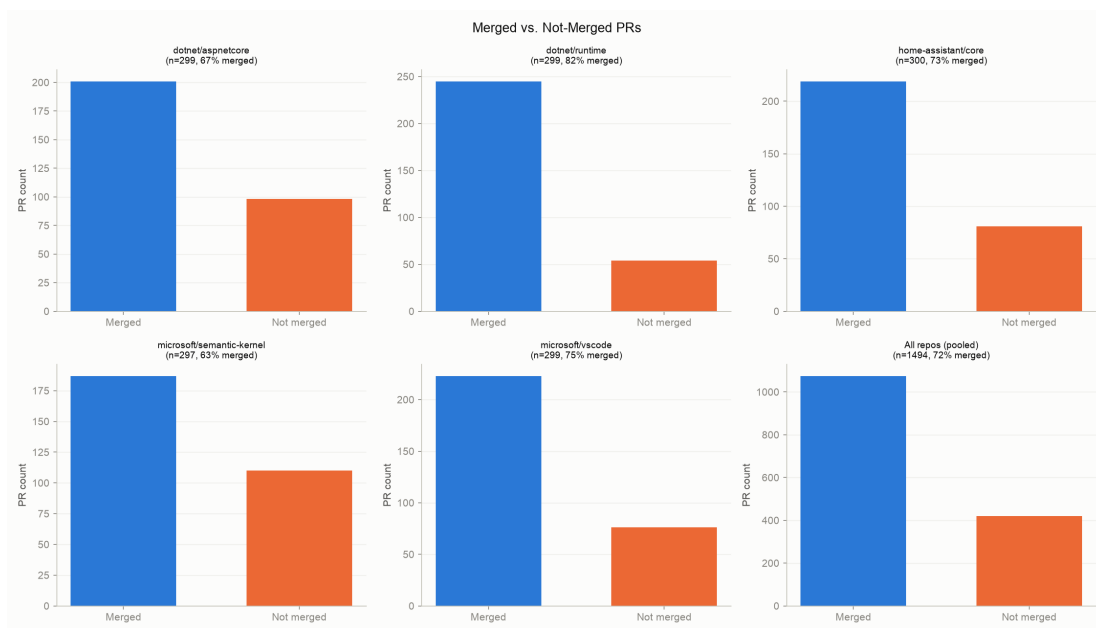


Figure 1. Merged vs. not-merged PR counts, per repo and pooled. Confirms the 72.0% pooled merge rate and shows every repo individually clears majority-merged, with `semantic-kernel` the closest to balanced (63.0%/37.0%).

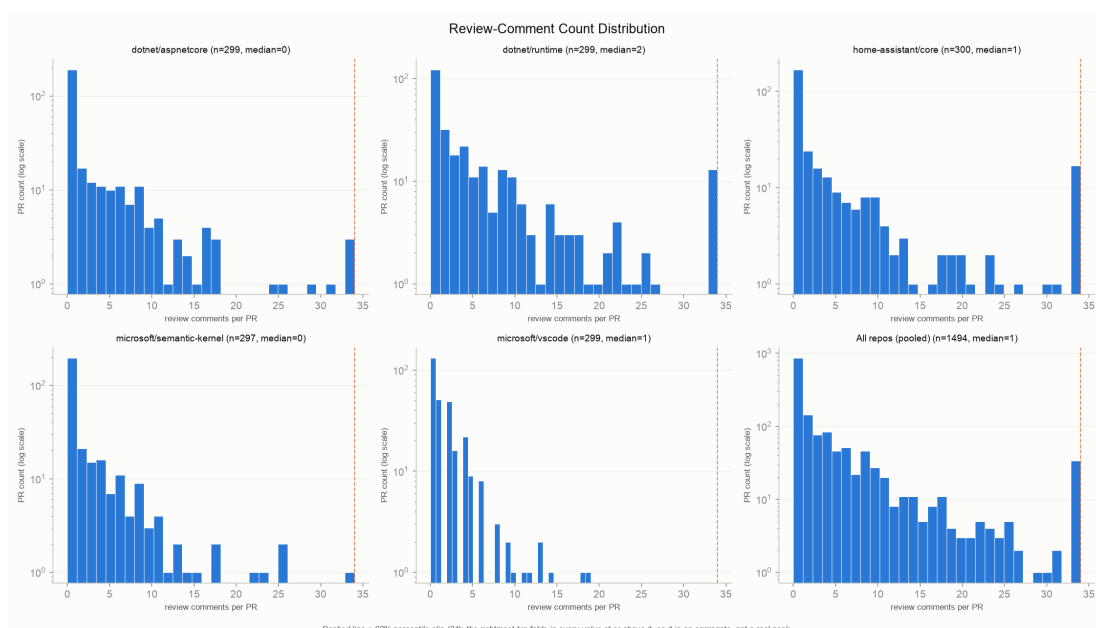


Figure 2. Review-comment-count distribution (log-scale y, clipped at the 98th percentile with the clip boundary marked — the rightmost bin is an aggregate of extreme values, not a real peak). Confirms the heavy right skew and the near-half of PRs with zero comments.

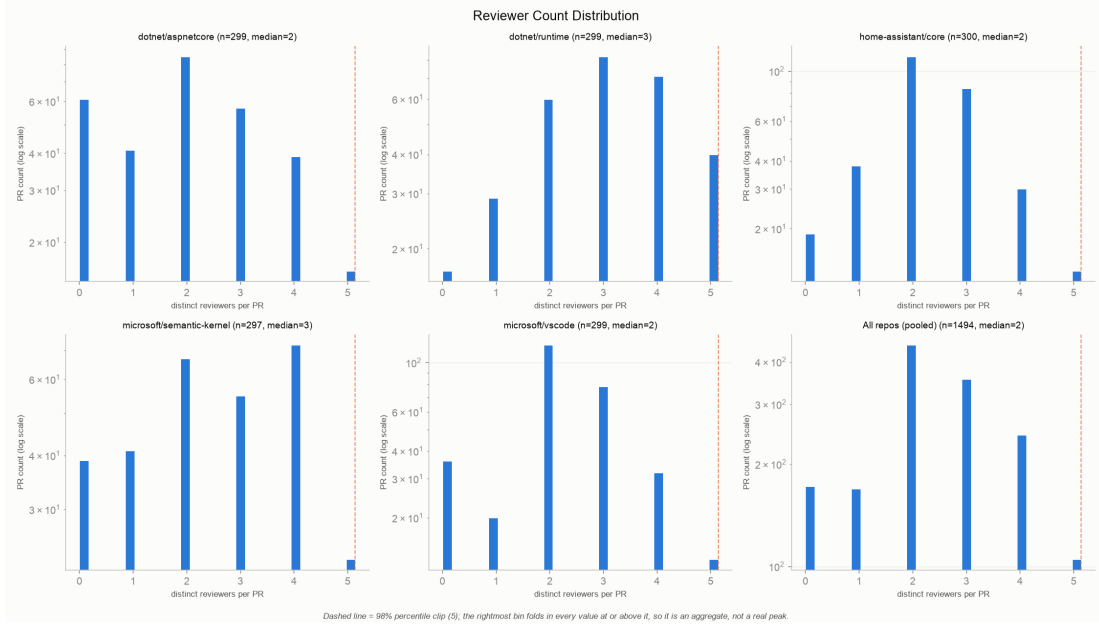


Figure 3. Reviewer-count distribution per repo and pooled; most PRs cluster at 1–3 distinct reviewers.

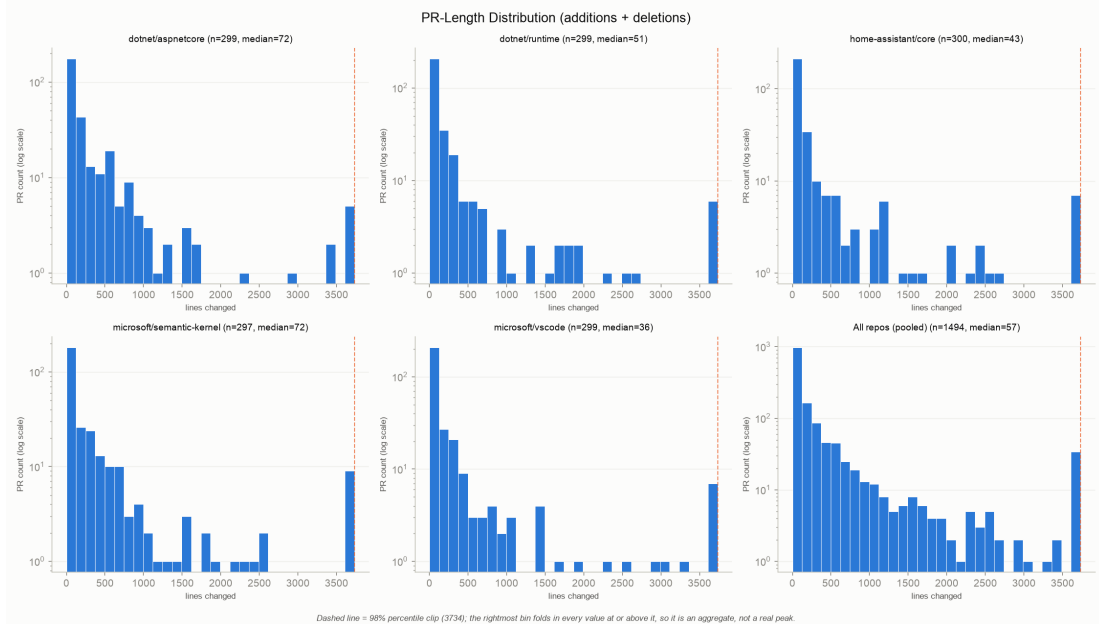


Figure 4. PR-length (additions + deletions) distribution, clipped and annotated identically to Figure 2. Medians in the tens of lines across every repo, confirming most PRs are small regardless of repository.

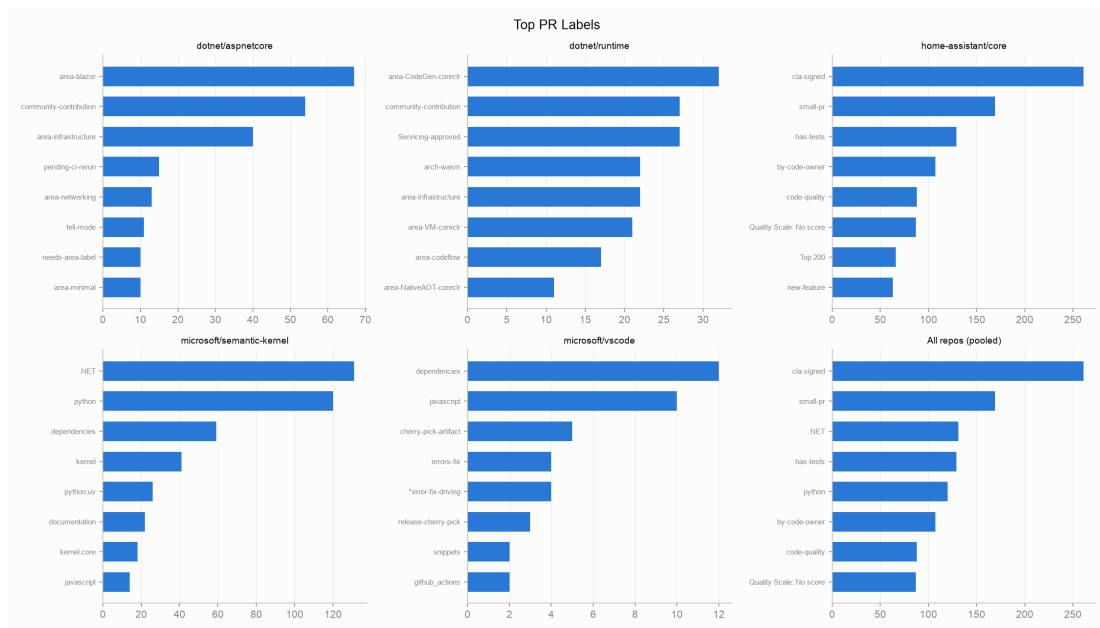


Figure 5. Top PR labels per repo and pooled. Visually confirms label usage is a per-repository convention: `home-assistant/core`'s labels dominate the pooled panel purely because that repo labels far more heavily, not because those labels are meaningful project-wide.

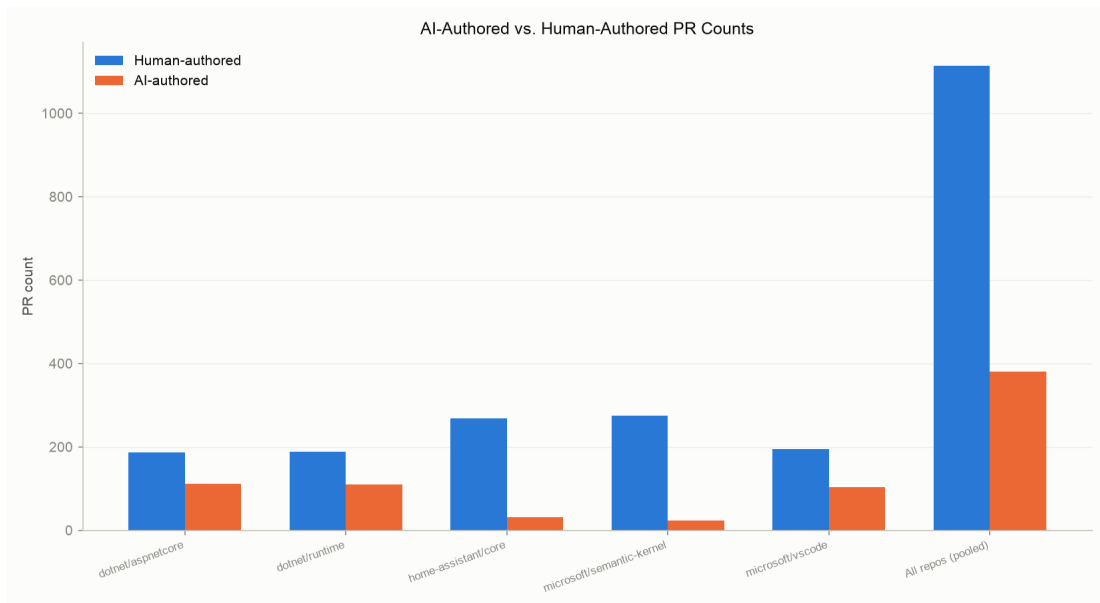


Figure 6. AI-authored vs. human-authored PR counts per repo. Visually confirms the large gap in AI-agent adoption between the two dotnet repos and `microsoft/vscode` versus `home-assistant/core` and `semantic-kernel`.

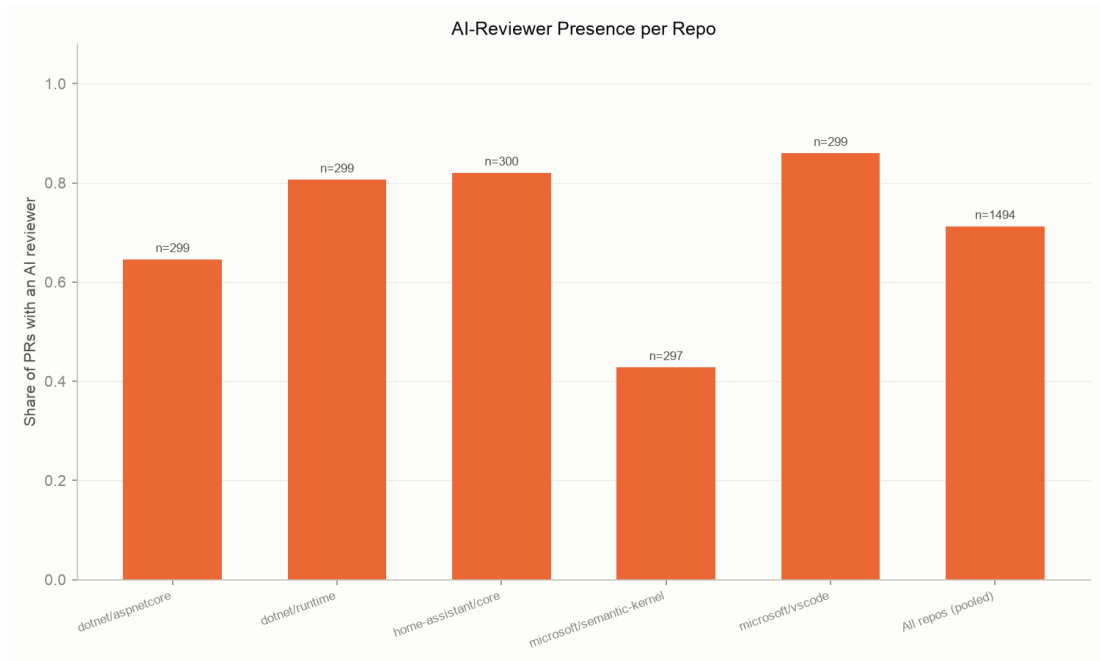


Figure 7. Share of PRs with an AI reviewer present, per repo, with sample size annotated on every bar.

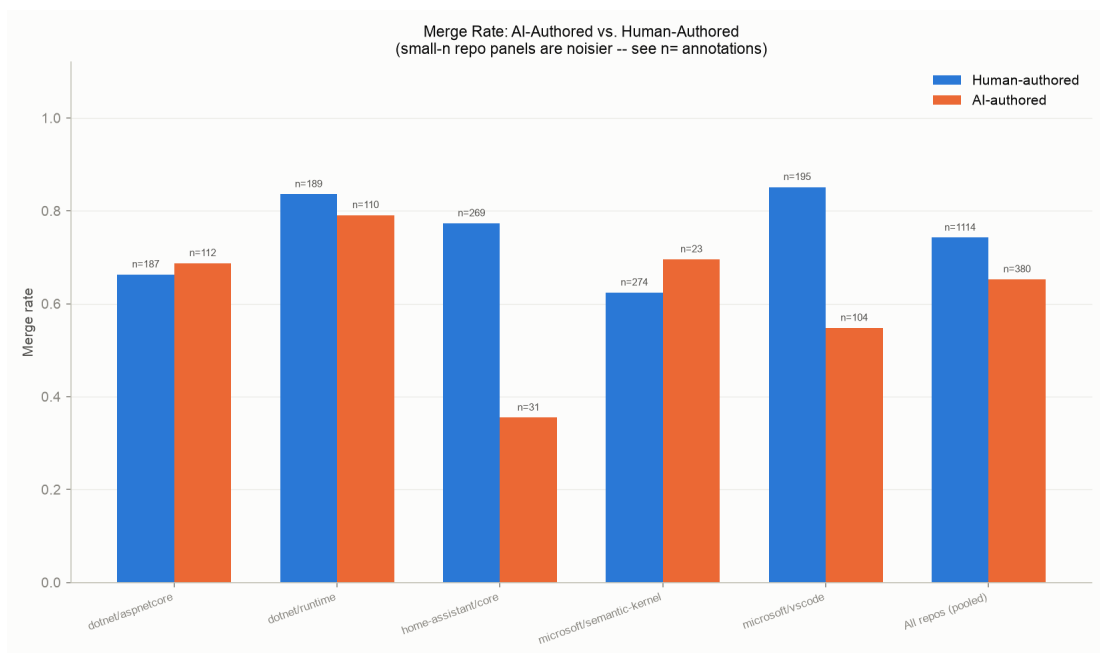


Figure 8. Merge rate, AI-authored vs. human-authored, per repo and pooled, with n annotated on every bar so the noisier small-sample repo panels (`semantic-kernel` $n = 23$, `home-assistant/core` $n = 31$) are not read with the same confidence as the pooled bar. This is the figure that makes the direction-flip described under “Statistical Results” visible at a glance.

4 Summary of Problems Encountered

Requirement: Primarily explain how the problems encountered during the experiment were resolved.

1. **GitHub secondary rate limiting and transient server errors during mining.** Fetching $\sim 1,500$ PRs concurrently occasionally triggered HTTP 403 (secondary rate limit), 429, and 502 responses. *Resolution:* exponential backoff with jitter, capped at 6 retry attempts; GraphQL-level rate-limit errors (returned as HTTP 200 with an `errors` array) were detected and retried separately from HTTP-level errors. 5 of 1,500 targeted PRs still failed after exhausting retries (all confirmed genuine server-side 502s) and were excluded and documented rather than retried indefinitely.
2. **Pagination-drift duplicate.** `microsoft/vscode#326842` was fetched and recorded twice. *Root cause:* the “recent closed PRs” fill listing sorts by `updated_at` on a live, high-traffic repository; an item can shift position between two sequential page fetches and land on both pages. *Resolution:* defensive deduplication was added at two layers — the sampling logic (`build_sample_numbers`) and the fetch layer (`GitHubClient.fetch_prs`) — and the already-mined dataset was rebuilt cleanly from the existing on-disk cache (no new network calls needed, since every PR was already fetched at least once).
3. **Schema gap masking genuinely distinct review comments as duplicates.** The `review_comments` and `issue_comments` tables initially did not carry the comment’s own GraphQL node ID, so 12 rows with identical visible fields (same PR, same rounded timestamp, similar body text) looked like accidental duplicates. *Investigation:* the raw cached JSON showed these were in fact three *genuinely distinct* comment nodes — GitHub’s Copilot reviewer had left three near-identical “missing type annotation” comments on three different parameters in the same file. *Resolution:* added a `comment_id` column to both tables, and added a `--rebuild-from-cache` mode to the mining script so all seven tables could be regenerated from the disk cache without re-mining.
4. **Missing LaTeX toolchain / `ctexart.cls`.** The report template’s class file loads `ctexart`, a Chinese-typesetting class from the full `ctex` bundle, which is unnecessary for a report written entirely in English and was not guaranteed to be present in a minimal LaTeX install. *Resolution:* a small local `ctexart.cls` shim (`reports/ctexart.cls`) subclasses plain `article` and defines only the handful of

commands `experimentreport.cls` actually calls (`\zihao{n}` as a font-size mapping, `\heiti` as `\bfseries`, `\chinese{counter}` as `\arabic{counter}`), avoiding the need to install real CJK support.

5 Reflections on the Experiment

(1) What are the differences between a Pull Request and a commit?

A commit is an atomic, immutable snapshot of a set of file changes with a message and an author, identified by a content hash (`sha`) — it is a Git-level object with no notion of review or merge status. A Pull Request is a GitHub-level, mutable social/workflow object that *groups one or more commits* together for review: it has its own title, description, labels, reviewers, discussion threads, and a final merge/close decision, and it can be updated with new commits after opening. In our data model this is exactly reflected in the table relationships: each row in `pull_requests` is a single mutable review unit, while `commits` is a child table keyed by `pr_id` with a many-to-one relationship — our 1,494 mined PRs carry 6,861 commits between them, an average of 4.59 commits per PR, and that ratio varies by repository (PRs get amended with follow-up commits addressing review feedback, which a bare commit has no equivalent of).

(2) Why does code review improve software quality?

Code review improves quality through several mechanisms visible in our own data: it catches defects before they reach the main branch (a second pair of eyes reading unfamiliar code finds issues the author, anchored to their own mental model, tends to miss); it enforces consistency (e.g. the observed human reviewer comment “the same issue at line 79 (for consistency)” is literally a consistency-enforcement act); and it transfers knowledge across the team (review threads are a running record of design rationale). Our AI-reviewer example — flagging a concrete `StopIteration` crash risk from a missing dictionary key — shows the same defect-catching mechanism operating automatically at scale: 71.2% of our mined PRs had at least one AI reviewer touch them, meaning a large fraction of this dataset now gets an automated first-pass review layer in addition to human review, not instead of it.

(3) In what practical scenarios can merge prediction be applied?

Concretely: (a) *maintainer triage* — surfacing PRs predicted unlikely to merge for early, low-cost feedback to the contributor rather than a long silent wait; (b) *CI/CD resource allocation* — prioritizing expensive test/build resources toward PRs likely to actually land; (c) *contributor-facing feedback tooling* — warning a contributor before they even open a PR that similar changes historically merge at a low rate in this repository; (d) *backlog health monitoring* — our own per-repo merge rates already range from 63.0%

to 81.9%, so a maintainer team could use a predictive model to distinguish “this PR looks like our typical successful PR” from “this PR’s characteristics resemble our historically-rejected PRs,” which a flat repository-wide prior cannot do.

(4) What are the potential differences in review approaches between AI reviewers and human reviewers?

Our own retrieved examples show this concretely rather than abstractly. The AI reviewer comment (“Guard against missing `DEFAULT_CLOUD...` to avoid `StopIteration` and a crashed config flow”) is a single-shot, self-contained technical observation naming a specific failure mode — consistent with an LLM pattern-matching against the diff in isolation. The human comments in our sample are conversational turns embedded in back-and-forth (“Thanks for the feedback! I’ll remove it.”) or reference project-wide context beyond the current diff (“along with the same issue at line 79, for consistency” — a cross-reference a purely diff-scoped reviewer would need explicit context to make). This matches the broader pattern the AI-detection work surfaced: `copilot-pull-request-reviewer` is present on all 1,064 of our 1,064 AI-reviewed PRs, suggesting a fairly uniform, high-volume automated commenting style, whereas human review style varies by reviewer and often carries social/negotiating content that a diff-only signal would miss — directly motivating Experiment 3/4’s inclusion of PR description, commit message, and repository-level context beyond the raw diff.

(5) Does the dataset suffer from class imbalance? How can this be addressed?

Yes: the pooled merge rate is 72.0% merged vs. 28.0% not merged (1,075 vs. 419), and the imbalance is not even uniform — it ranges from 63.0% (`semantic-kernel`) to 81.9% (`dotnet/runtime`) by repository. This is a moderate but real imbalance: a trivial always-predict-merged classifier would already score 72.0% accuracy, making raw accuracy a misleading metric on its own. This will be addressed in Experiment 2 by: (a) using a time-based train/test split rather than a random split, to avoid look-ahead leakage in addition to the class-balance question; (b) training with `class_weight='balanced'` and comparing against the unweighted model; (c) reporting precision/recall/F1 per class — specifically minority-class (not-merged) recall — rather than accuracy alone, since accuracy alone would let a model that never predicts “not merged” look artificially strong on this dataset.

Instructor’s Comments:

Date: